

重慶理工大學

物联网通信技术设计报告

二 级 学 院 计算机科学与工程学院

专　　业 物联网工程

班　　级

学 生 姓 名　　学号

题　　目 校园热水站数据采集通信协议设计报告

评　　语

系统设计（50%）	课堂答辩及报告（50%）	总评（100%）	总评教师

目 录

摘 要 II
第一章 项目概述 1
第二章 需求分析 2
2.1 用户需求分析 2
2.2 设计需求（协议非功能性需求） 2
第三章 详细协议设计 4
3.1 协议概述 4
3.2 协议框架设计 4
3.3 协议报文格式 4
3.4 命令字定义 5
3.5 关键业务流程 7
3.6 数据格式定义 9
3.7 安全设计 9
3.8 错误处理 10
3.9 性能指标 11
3.10 扩展性设计 11
附 录 13

摘　要

本报告设计了一套用于校园热水站智能监控系统的专用通信协议，旨在解决大规模、分布式热水设备的数据采集、状态监控与集中管理问题。协议面向低功耗、窄带宽的物联网场景（如 4G/NB-IoT），采用主从式长连接架构，定义了精简高效的二进制报文格式、完备的通信流程及可靠的安全机制。

关键词：通信协议、数据帧、热水站（3-5 个）

第一章　项目概述

随着高校后勤管理数字化、智能化转型的深入，传统的校园热水供应系统（如澡堂、开水房、宿舍淋浴间）面临诸多挑战：计费不透明、故障响应慢、数据统计难、能源浪费大、用户体验差。为解决这些问题，建设基于物联网（IoT）的智能热水供应系统已成为必然趋势。

本报告旨在定义该系统中最为核心的组成部分——**智能水表（终端）与集中器（云平台之间）**的通信协议。该协议是确保数据准确采集、指令可靠下发、系统稳定运行的关键，是实现智能计量、远程管控、数据分析的基础。

第二章　需求分析

2.1　用户需求分析

用户需求主要来源于系统最终使用者（学生）、运营管理者（后勤部门）和维护人员。

A. 学生（最终用户）需求：

透明计费：实时、准确地显示用水量与消费金额，消费记录可便捷查询，杜绝“糊涂账”。

稳定可靠：刷卡或扫码后迅速出水，用水过程中无中断，结束响应及时。

便捷支付：支持校园一卡通、微信/支付宝扫码、APP 充值等多种支付方式。

故障反馈：遇到设备故障（如不出水、关不停）时，能有简便渠道快速报修。

节能省费：可自主查询用水习惯，获取节能建议。

B. 后勤运营管理需求：

精准计量与收费：实现按实际用水量计费，杜绝包月制下的浪费，提升经济效益。

实时监控：远程实时监控所有热水终端的在线状态、工作状态（空闲、使用中、故障）、累计用水量等。

远程控制：可对指定终端进行远程开关阀、费率调整、设备重启等操作。

财务与数据分析：自动生成各类财务报表（日/月/年报表、单设备报表）、用水量分析（高峰时段、人均用水）、能耗分析，为决策提供支持。

故障预警与告警：设备离线、阀门异常、通讯中断、低余额、传感器故障等能主动上报，并定位到具体设备，实现预测性维护。

参数统一下发：能够批量对终端进行费率、时钟、开关机时间等参数同步。

C. 系统维护人员需求：

快速定位：通过协议返回的详细状态码和日志，能快速定位硬件或网络问题。

远程诊断与升级：支持远程查询终端参数、日志，并支持固件远程升级（FOTA）。

安装调试简便：协议应支持设备的自注册、自动网络接入，降低现场配置复杂度。

2.2　设计需求（协议非功能性需求）

基于用户需求，推导出协议本身必须满足的设计需求：

可靠性：在校园复杂的无线（如 NB-IoT、LoRa、Wi-Fi）或有线环境下，保证数

据传输的准确、不丢失、不重复。关键指令（如扣费、关阀）需具备确认机制。

实时性：状态变化（如开始用水、结束用水）应在数秒内上报，控制指令应在可接受延时内（如 <5 秒）得到执行。

安全性：

数据保密性：对关键用户数据（如用户 ID、交易记录）进行加密传输。

数据完整性：使用报文校验（如 CRC16）和数字签名，防止数据在传输中被篡改。

身份认证：终端与服务器之间需进行双向认证，防止非法设备接入或伪服务器攻击。

防重放攻击：重要交易指令需包含序列号或时间戳，防止指令被恶意重复发送。

低功耗：对于采用电池供电的无线终端，协议应支持心跳保活、低功耗睡眠模式，仅在必要时刻通讯，以延长设备寿命。

可扩展性：协议帧结构应预留字段，支持未来新增功能（如水温、水压监测）而不影响旧版本兼容性。

易解析与高效性：报文结构应紧凑、清晰，减少冗余，以节省流量（特别是蜂窝网络）和提高解析效率。

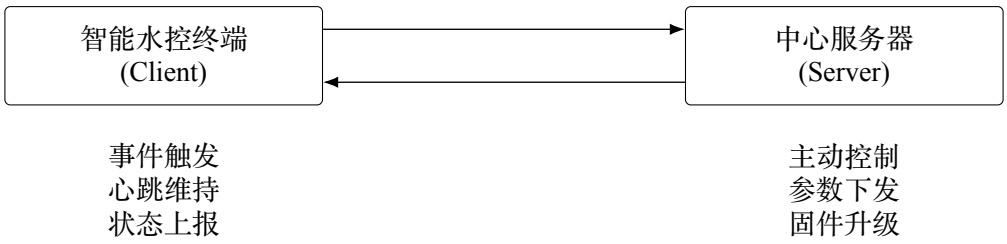
第三章　详细协议设计

3.1　协议概述

设计原则：本协议采用分层设计思想，分为传输层适配和应用层协议。应用层协议独立于底层网络，可适配 TCP/IP、UDP/CoAP 等多种传输方式。协议设计遵循“请求-响应”与“主动上报”相结合的模式，兼顾实时性和可靠性。

3.2　协议框架设计

3.2.1　通信模型



3.2.2　连接方式

主连接：TCP 长连接（有线/Wi-Fi）或 CoAP/UDP（NB-IoT/LoRa）

保活机制：心跳包维持连接，心跳间隔可配置（默认 300 秒）

断线重连：设备检测到断线后按指数退避算法重连

3.2.3　通信模式

设备主动上报：用水事件、故障告警、状态变化

服务器请求-设备响应：服务器查询设备状态、参数

服务器指令-设备执行：远程控制、参数配置

设备请求-服务器响应：心跳、注册、校时

3.3　协议报文格式

3.3.1　基本帧结构

帧头 **(2)**	版本 **(1)**	命令字 **(1)**	序列号 **(2)**	设备 **ID(8)**	长度 **(2)**	负载 **(N)**	校验 **(2)**
0xAA55	0x01	Cmd	Seq	DevID	Len	Data	CRC

3.3.2 字段详细说明

字段名	长度	描述	备注
帧头	2 字节	固定值 0xAA55	用于帧同步
协议版本	1 字节	主版本号 (高 4 位)+ 次版本号 (低 4 位)	版本 1.2 → 0x12
命令字	1 字节	指令类型标识	见 3.4 节命令字定义
序列号	2 字节	消息序号，从 0 递增，循环使用	用于请求-响应匹配和防重放
设备 ID	8 字节	设备唯一标识符	生产时烧录，格式: 厂商 (2)+ 型号 (2)+ 序号 (4)
负载长度	2 字节	负载数据的字节长度	大端字节序，范围 0-1024
负载数据	N 字节	具体业务数据	结构随命令字变化
CRC16 校验	2 字节	对整个帧（除校验字段）的 CRC16 校验	多项式: 0x8005

3.4 命令字定义

3.4.1 基础命令集（**0x00-0x3F**）

命令字	名称	方向	说明
0x01	心跳上报	设备 → 服务器	设备在线状态报告
0x02	心跳响应	服务器 → 设备	服务器确认收到心跳
0x03	设备注册	设备 → 服务器	设备首次上线注册
0x04	注册响应	服务器 → 设备	注册结果，下发设备密钥
0x05	时间同步请求	设备 → 服务器	请求服务器时间
0x06	时间同步响应	服务器 → 设备	返回服务器时间戳

3.4.2 用水控制命令集（0x10-0x1F）

命令字	名称	方向	说明
0x10	用水开始事件	设备 → 服务器	用户刷卡/扫码成功，开始用水
0x11	用水结束事件	设备 → 服务器	用户停止用水或超时停止
0x12	实时用水上报	设备 → 服务器	用水过程中定时上报累积水量
0x13	远程开阀指令	服务器 → 设备	管理员远程开阀
0x14	远程关阀指令	服务器 → 设备	管理员远程关阀
0x15	开关阀响应	设备 → 服务器	执行开关阀操作的结果

3.4.3 状态查询命令集（0x20-0x2F）

命令字	名称	方向	说明
0x20	设备状态查询	服务器 → 设备	查询设备当前状态
0x21	设备状态上报	设备 → 服务器	上报设备状态信息
0x22	累计数据查询	服务器 → 设备	查询设备累计用水量等信息
0x23	累计数据上报	设备 → 服务器	上报累计数据
0x24	实时数据查询	服务器 → 设备	查询设备实时数据（温度、流量等）
0x25	实时数据上报	设备 → 服务器	上报实时数据

3.4.4 参数配置命令集（0x30-0x3F）

命令字	名称	方向	说明
0x30	参数读取请求	服务器 → 设备	读取设备参数
0x31	参数读取响应	设备 → 服务器	返回设备参数
0x32	参数设置指令	服务器 → 设备	设置设备参数
0x33	参数设置响应	设备 → 服务器	参数设置结果
0x34	费率参数下发	服务器 → 设备	下发费率表
0x35	费率参数确认	设备 → 服务器	费率接收确认

3.4.5 故障告警命令集（0x40-0x4F）

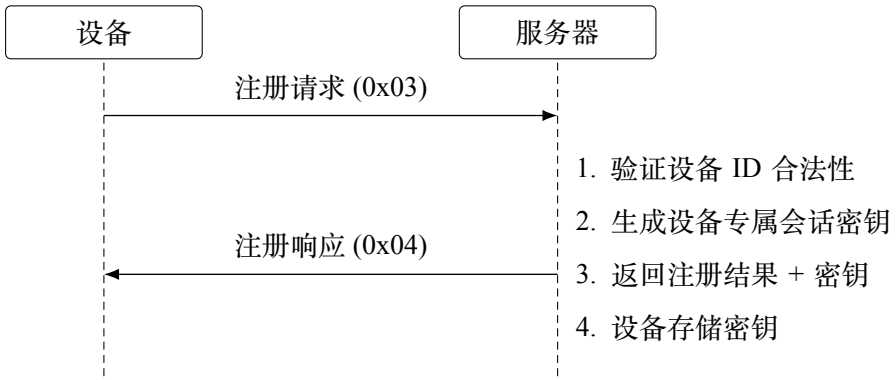
命令字	名称	方向	说明
0x40	故障实时上报	设备 → 服务器	故障发生时立即上报
0x41	故障确认指令	服务器 → 设备	服务器确认收到故障
0x42	故障恢复上报	设备 → 服务器	故障恢复时上报
0x43	告警查询指令	服务器 → 设备	查询设备告警记录
0x44	告警记录上报	设备 → 服务器	上报告警记录

3.4.6 固件升级命令集（0x50-0x5F）

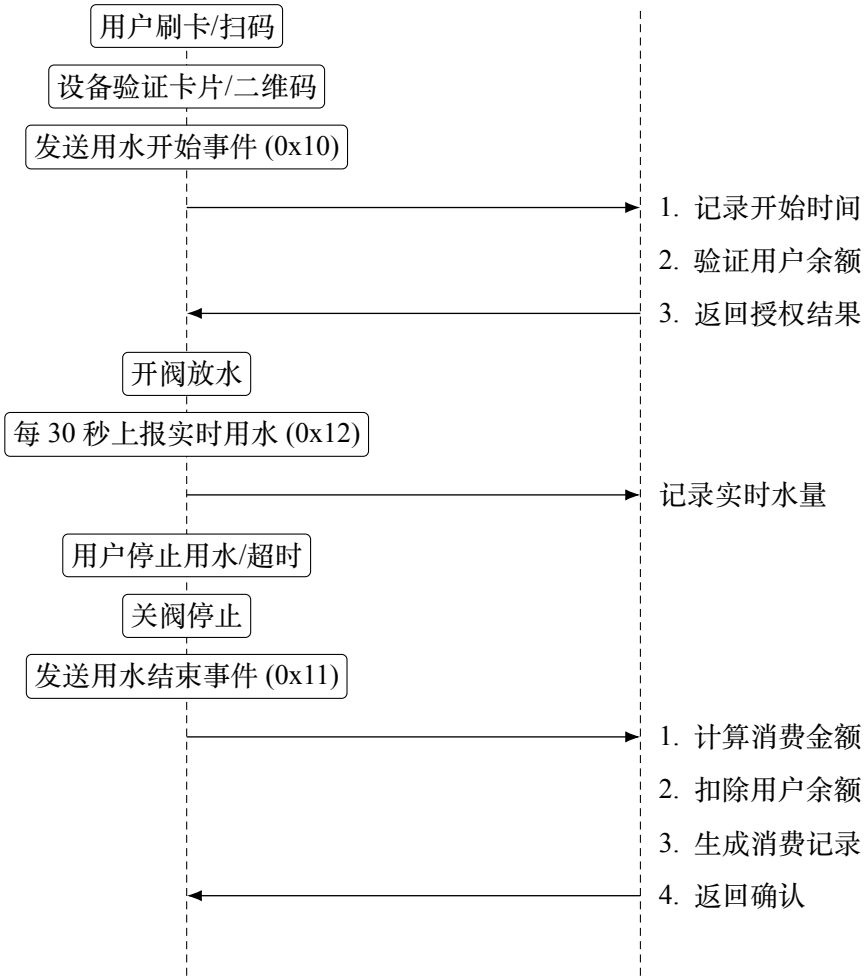
命令字	名称	方向	说明
0x50	升级通知指令	服务器 → 设备	通知设备有新固件
0x51	升级请求	设备 → 服务器	设备请求升级
0x52	升级数据包	服务器 → 设备	下发固件数据包
0x53	升级确认	设备 → 服务器	数据包接收确认
0x54	升级结果上报	设备 → 服务器	升级成功/失败结果

3.5 关键业务流程

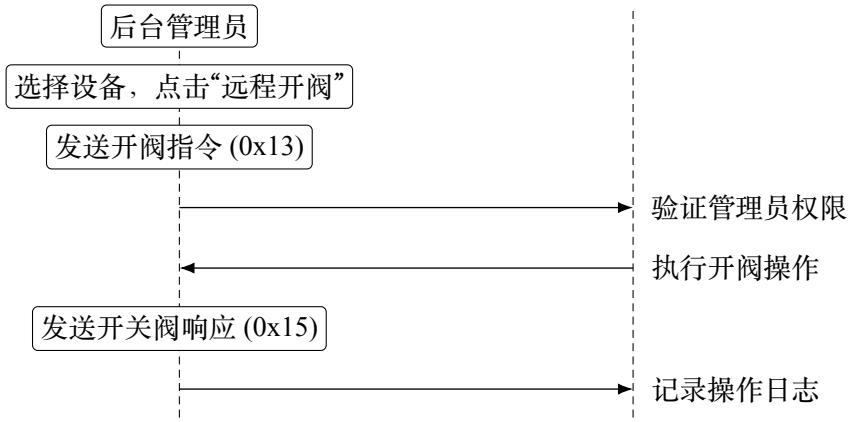
3.5.1 设备注册流程



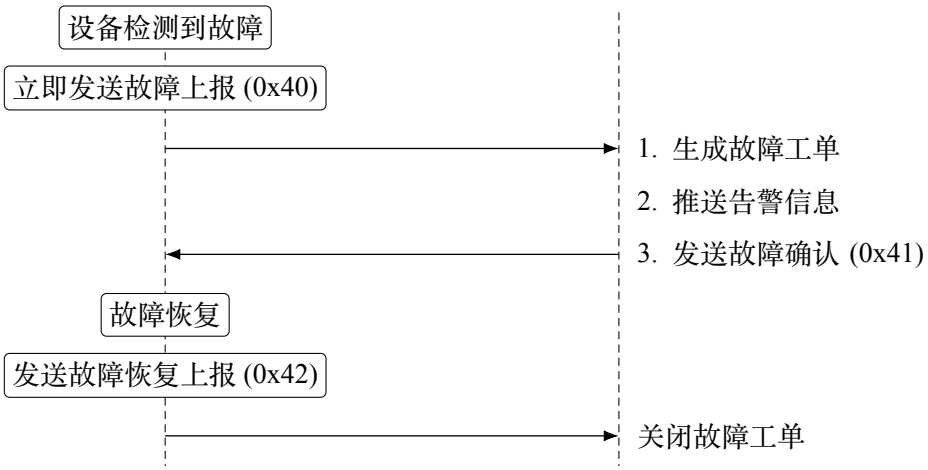
3.5.2 用水控制流程



3.5.3 远程控制流程



3.5.4 故障上报流程



3.6 数据格式定义

3.6.1 用水开始事件（**0x10**）负载格式

用户 ID(8)	卡片类型	余额 (4)	费率 ID(2)	预留 (2)	时间戳 (4)

3.6.2 用水结束事件（**0x11**）负载格式

用户 ID (8)	结束水量 (4)	用水时长 (4)	消费金额 (4)	原因 (1)	预留 (3)	时间戳 (4)

3.6.3 设备状态（**0x21**）负载格式

阀门状态 (1)	水温 (2)	水压 (2)	信号强度 (1)	电压 (2)	故障码 (2)	工作模式 (1)	预留 (2)

3.6.4 故障上报（**0x40**）负载格式

故障代码 (2)	故障级别 (1)	故障数值 (4)	相关参数 1 (4)	相关参数 2 (4)	预留 (2)	时间戳 (4)

3.7 安全设计

3.7.1 加密方案

传输层：强制使用 TLS 1.2+（TCP 连接）或 DTLS（UDP 连接）

应用层：敏感数据使用 AES-128-GCM 加密

密钥管理：设备出厂预置根密钥，注册时更新为会话密钥

3.7.2 认证机制

设备认证：基于预置证书/密钥的 TLS 双向认证

消息认证：每条消息携带基于会话密钥的 HMAC-SHA256

用户认证：刷卡/扫码时与后台系统交互认证

3.7.3 防重放攻击

每个消息包含递增序列号（2 字节，循环使用）

服务器维护设备最近 N 个序列号的滑动窗口

时间戳校验，允许 ±5 分钟的时间误差

3.7.4 数据完整性

帧级：CRC16 校验

消息级：HMAC 消息认证码

应用级：关键业务数据包含数字签名

3.8 错误处理

3.8.1 错误码定义

错误码	含义	处理建议
0x00	成功	正常处理
0x01	协议版本不支持	升级设备固件
0x02	命令字不支持	检查协议版本
0x03	数据格式错误	检查数据格式
0x04	参数错误	检查参数范围
0x05	认证失败	重新注册或检查密钥
0x06	权限不足	检查操作权限
0x07	设备忙	稍后重试
0x08	内存不足	清理设备数据
0x09	网络异常	检查网络连接
0x0A	服务器忙	稍后重试
0x0B-0x7F	预留	

3.8.2 重传机制

关键指令（扣费、控制）：最多重试 3 次，间隔 2 秒、4 秒、8 秒

普通数据：最多重试 1 次

心跳包：不重传，超时直接重连

3.9 性能指标

3.9.1 通信性能

指标	目标值	备注
事件上报延迟	<3 秒	用水开始/结束
控制指令响应	<5 秒	远程开关阀
状态查询响应	<2 秒	实时状态查询
心跳间隔	300 秒	可配置
最大包大小	1024 字节	固件升级除外

3.9.2 网络流量估算

场景	上行流量	下行流量	频率
心跳包	30 字节	20 字节	每 5 分钟
用水开始事件	50 字节	20 字节	每次用水
用水结束事件	60 字节	20 字节	每次用水
实时用水上报	40 字节	0 字节	每 30 秒（用水中）
月均用量估算	~5MB	~2MB	按每天 50 次用水

3.10 扩展性设计

3.10.1 版本兼容

主版本号不同：不兼容，拒绝服务

次版本号不同：向后兼容，忽略未知字段

扩展字段使用 TLV 格式，旧版本可跳过未知类型

3.10.2 功能扩展预留

负载预留字段：每个消息负载保留 4 字节扩展位

命令字空间：0x80-0xFF 为厂商自定义命令

设备能力协商：注册时上报支持的功能列表

3.10.3 未来可扩展功能

水温自适应调节（需增加温度传感器）

用水习惯分析（需增加数据分析字段）

语音提示功能（需增加语音控制命令）

多人用水模式（需增加多用户管理）

附　录

1 数据类型定义

```
// 基本数据类型
typedef uint8_t  BYTE;       // 1字节无符号
typedef uint16_t WORD;       // 2字节无符号
typedef uint32_t DWORD;      // 4字节无符号
typedef uint64_t QWORD;      // 8字节无符号
typedef int32_t  LONG;       // 4字节有符号

// 时间类型：Unix时间戳（秒）
typedef DWORD TIMESTAMP;

// 金额类型：单位分（1元=100分）
typedef LONG MONEY;
```

2 故障码表（部分）

0x0001: 阀门故障 - 开阀失败

0x0002: 阀门故障 - 关阀失败

0x0003: 流量计故障 - 无脉冲信号

0x0004: 温度传感器故障

0x0005: 水压传感器故障

0x0006: 存储器故障

0x0007: 电源电压过低

0x0008: 通讯模块故障

0x0009: 余额不足警告

0x000A: 非法卡警告

3 协议示例

```
# 用水开始事件示例（0x10）
frame = {
    'header': 0xAA55,
    'version': 0x01,
    'cmd': 0x10,
```

```
    'seq': 0x0102,
    'dev_id': 0x1234567890ABCDEF,
    'length': 24,
    'payload': {
        'user_id': 0x1001,
        'card_type': 1,      # 1:校园卡，2:微信，3:支付宝
        'balance': 5000,     # 50.00元
        'rate_id': 1,
        'reserved': 0,
        'timestamp': 1672502400
    },
    'crc': 0x1234
}
```

4 CRC-16 计算示例

示例 1：心跳包（命令字 0x01）

场景：设备 ID 为 0x1122334455667788 的设备发送心跳包

步骤 1：构建数据包（不含 CRC）

原始数据（16 进制表示）：

```
AA 55 01 01 01 02 11 22 33 44 55 66 77 88 00 00
```

字段解析：

```
AA 55                   - 帧头 (0xAA55)
01                      - 版本 (V1.1)
01                      - 命令字 (0x01 = 心跳)
01 02                   - 序列号 (0x0102)
11 22 33 44 55 66 77 88 - 设备 ID
00 00                   - 负载长度 (0 字节，无负载)
```

步骤 2：计算 CRC

```
data = bytes.fromhex('AA55010101021122334455667788000')
crc = CRC16_MODBUS.calculate(data)
print(f"CRC16校验值: 0x{crc:04X}")  # 输出: 0xB9A5
```

步骤 3：添加 CRC 得到完整数据包

完整数据包（16 进制）：

```
AA 55 01 01 01 02 11 22 33 44 55 66 77 88 00 00 B9 A5
```

字节位置说明：

```
00-01: 帧头      0xAA55
02   : 版本      0x01
03   : 命令字    0x01
04-05: 序列号    0x0102
06-13: 设备 ID   0x1122334455667788
14-15: 长度      0x0000
16-17: CRC16     0xB9A5
```

步骤 4：服务器验证 CRC

```
# 服务器端验证
full_packet = bytes.fromhex('AA55010101021122334455667788000B9A5')
is_valid = CRC16_MODBUS.verify(full_packet)
if is_valid:
    print("CRC验证通过")
else:
    print("CRC验证失败，数据可能被篡改")
```